

2.2 Guía de Funciones: Estadísticos Descriptivos y Limpieza

R — dplyr + base R

```
library(dplyr)

# ESTADÍSTICOS DESCRIPTIVOS
# =====
df <- data.frame(
  valor = c(10, 20, NA, 35, 200, 15, 22, 18, 25, 30),
  grupo = c("A", "A", "B", "B", "A", "B", "A", "B", "A", "B")
)

# Resumen completo
summary(df)
# → Min, Q1, Mediana, Media, Q3, Máx; NAs contados

# Estadísticos individuales
mean(df$valor, na.rm = TRUE) # media (ignora NA)
median(df$valor, na.rm = TRUE) # mediana
sd(df$valor, na.rm = TRUE) # desvío estándar
var(df$valor, na.rm = TRUE) # varianza
IQR(df$valor, na.rm = TRUE) # rango intercuartílico
quantile(df$valor, probs = c(0.25, 0.75), na.rm = TRUE) # Q1 y Q3

# Detección de outliers (regla de Tukey)
Q1 <- quantile(df$valor, 0.25, na.rm=TRUE)
Q3 <- quantile(df$valor, 0.75, na.rm=TRUE)
IQR_val <- Q3 - Q1
outliers <- df$valor[df$valor < Q1 - 1.5*IQR_val |
  df$valor > Q3 + 1.5*IQR_val]
# na.rm=TRUE: ignora NAs en el cálculo
```

```
# LIMPIEZA DE DATOS
# -----

# Detectar NAs
sum(is.na(df$valor))      # total de NAs
colSums(is.na(df))        # NAs por columna

# Eliminar filas con NA
df_sin_na <- na.omit(df)

# 0 con dplyr:
df_sin_na <- df %>% filter(!is.na(valor))

# Imputar con la mediana del grupo
df_imputado <- df %>%
  group_by(grupo) %>%
  mutate(valor = ifelse(is.na(valor), median(valor, na.rm=TRUE), valor)) %>%
  ungroup()

# group_by: agrupa para imputar por grupo
# ifelse: si es NA, reemplaza con mediana del grupo

# Eliminar duplicados
df_unico <- df %>% distinct()
# distinct(columna1, columna2): duplicados basados en columnas específicas

# Cambiar tipos
df$grupo <- as.factor(df$grupo)  # texto → factor
df$valor <- as.numeric(df$valor) # texto → número

# Normalización min-max
normalizar <- function(x) (x - min(x, na.rm=T)) / (max(x, na.rm=T) - min(x, na.rm=T))
df$valor_norm <- normalizar(df$valor)

# Estandarización Z-score
df$valor_z <- scale(df$valor)    # media=0, sd=1
```

Python — pandas

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
  'valor': [10, 20, None, 35, 200, 15, 22, 18, 25, 30],
  'grupo': ['A', 'A', 'B', 'B', 'A', 'B', 'A', 'B', 'A', 'B']
})
```

```
})

# ESTADÍSTICOS DESCRIPTIVOS
df['valor'].describe()      # resumen completo
df['valor'].mean()          # media
df['valor'].median()        # mediana
df['valor'].std()           # desvío estándar
df['valor'].quantile([.25,.75]) # Q1, Q3

# Outliers (Tukey)
Q1 = df['valor'].quantile(0.25)
Q3 = df['valor'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['valor'] < Q1 - 1.5*IQR) | (df['valor'] > Q3 + 1.5*IQR)]

# LIMPIEZA
df.isnull().sum()          # NAs por columna
df.dropna()                # eliminar filas con NA
df['valor'].fillna(df['valor'].median()) # imputar con mediana

# Imputar por grupo
df['valor'] = df.groupby('grupo')['valor'].transform(
    lambda x: x.fillna(x.median())
)
# transform: aplica función y devuelve serie del mismo tamaño

df.drop_duplicates()        # eliminar duplicados

# Normalización min-max
from sklearn.preprocessing import MinMaxScaler, StandardScaler
scaler_mm = MinMaxScaler()
df['valor_norm'] = scaler_mm.fit_transform(df[['valor']])

scaler_z = StandardScaler()
df['valor_z'] = scaler_z.fit_transform(df[['valor']])
```